ARIMA, FARIMA and ARIMA Models 3. Rough Guide

These three model families implement three complementary intuitions about specific temporal dependencies:

1. ARIMA. Constrained memory model. The few previous samples matter, but it does not remember too far back in time.
2. FARIMA. Even distant past events matter.
3. SARIMA. The pattern repeats with every season (e.g., monthly, yearly)

1. AutoRegressive Integrated Moving Average (ARIMA).

$$\phi(B)(1-B)^d y_t = \theta(B)\epsilon_t$$

- Lag Operator B : $By_t = y_{t-1}$
- Auto-regressor: $\phi(B) = 1 - \phi_1 B - ... - \phi_p B^p$
- Differencing : $(1-B)^d$
- Moving Average : $\theta(B) = 1 + \theta_1 B + ... + \theta_q B^q$
- White noise : ϵt

ARIMA models include three parameters:

- p - AR order.
- d – degree of differencing (d=0, no differencing; d=1, model uses changes between consecutive samples).
- q – MA order.

Example ARIMA(1,1,1):

$$y_t - y_{t-1} = \phi_1(y_{t-1} - y_{t-2}) + \epsilon_t + \theta_1 \epsilon_{t-1}$$

NB. ARIMA(1,1,1) does not model y, but their first difference.

Example 1. Stock returns with no-seasonality (ARIMA):

Stock returns with no-seasonality. We simulate a stationary ARIMA(1,0,1) process.

```
import numpy as np
import pandas as pd
from statsmodels.tsa.arima.model import ARIMA

# simulate data
np.random.seed(0)
n = 200
```

```
noise = np.random.normal(size=n)
y = [0]

for t in range(1, n):
    y.append(0.6*y[t-1] + noise[t] + 0.5*noise[t-1])

y = pd.Series(y)

# fit ARIMA(1,0,1)
model = ARIMA(y, order=(1,0,1))
result = model.fit()

print(result.summary())
```

Conclusion: there is a short-term dependence, which dies out rapidly.

2. Fractional ARIMA/ARFIMA (FARIMA).

$$\phi(B)(1-B)^d y_t = \theta(B)\epsilon_t, \quad d \in \mathbb{R}$$

The key difference ist hat d is fractional (e.g., 0.3 instead of 1), which creates a long-term memory with slow decaying autocorrelation.

Example 2. Long-memory (FARIMA)

`The statsmodel` library has limited direct FARIMA support, so we simulate fractional differencing manually.

```
import numpy as np
import pandas as pd
from statsmodels.tsa.arima.model import ARIMA

def fractional_noise(d, n):
    w = np.random.normal(size=n)
    x = np.zeros(n)
    for t in range(n):
        for k in range(t+1):
            coef = np.math.gamma(k - d) / (np.math.gamma(-d) *
np.math.gamma(k + 1))
            x[t] += coef * w[t-k]
    return x

np.random.seed(0)
n = 200
d = 0.3

y = fractional_noise(d, n)
y = pd.Series(y)

# approximate with ARIMA (since FARIMA not directly supported)
model = ARIMA(y, order=(1,0,1))
result = model.fit()
```

```
print(result.summary())
```

Conclusion: Even distant past values influence the present.

3. SARIMA (Seasonal ARIMA)

$$\phi(B)\Phi(B^s)(1-B)^d(1-B^s)^D y_t = \theta(B)\Theta(B^s)\epsilon_t$$

Where:

- s: seasonal period (e.g., 12 for monthly data)
- $\Phi(B^s), \Theta(B^s)$: seasonal AR and MA parts
- $(1-B^s)^D$: seasonal differencing

Example 3. Monthly seasonal data (SARIMA)

Start by simulating data with with yearly seasonality (period = 12).

```
import numpy as np
import pandas as pd
from statsmodels.tsa.statespace.sarimax import SARIMAX

np.random.seed(0)
n = 240
t = np.arange(n)

# create seasonal signal
seasonal = 10 * np.sin(2 * np.pi * t / 12)
noise = np.random.normal(scale=2, size=n)

y = seasonal + noise
y = pd.Series(y)

# fit SARIMA(1,0,1)(1,1,1,12)
model = SARIMAX(y, order=(1,0,1), seasonal_order=(1,1,1,12))
result = model.fit()

print(result.summary())
```

Conclusion: Captures repeating yearly structure + short-term dynamics.